Asistente Organizador Académico — Agente Funcional

Evaluación Parcial N°2 · ISY0101 Ingeniería de Soluciones con IA · Duoc UC

Integrantes: Juan Mondaca — Iván Álvarez **Profesor:** Francisco Macaya **Repositorio:** https://github.com/Valstrax69/AsistenteorganizadorIA **Octubre 2026**

---

1. Introducción

La EP1 entregó un chatbot RAG que respondía consultas sobre documentación institucional de Duoc UC. Esta segunda etapa lo convierte en un **agente funcional**: el modelo ya no solo redacta respuestas, sino que decide por sí mismo qué herramientas ejecutar para resolver lo que el estudiante le pide.

El cambio es concreto. Antes, el sistema recuperaba contexto y generaba texto. Ahora puede **consultar** el material institucional y la agenda del estudiante, **escribir** en esa agenda, y **razonar** sobre ella para priorizar la carga académica.

2. Arquitectura del agente

La orquestación sigue el patrón ReAct, implementado de forma explícita en `rag/agente.py`:

1. Se arma el contexto: prompt del sistema, memoria de largo plazo, historial reciente y la pregunta.
2. El modelo, enlazado a las herramientas mediante `bind_tools`, responde o solicita una o más herramientas.
3. Si solicita herramientas, se ejecutan y cada resultado vuelve al modelo como `ToolMessage`.
4. El ciclo se repite hasta que el modelo responde sin pedir nada más, o hasta agotar `MAX_PASOS_AGENTE`.

Componente	Tecnología	Rol
Framework	LangChain (`langchain-groq`)	`@tool` para declarar herramientas, `bind_tools` para exponerlas al modelo
LLM	Groq, `openai/gpt-oss-120b`	Decide qué herramienta llamar y redacta la respuesta
Embeddings	`multilingual-e5-small` (local, 384 dims)	Vectoriza documentos y consultas sin costo por token
Base vectorial	MongoDB Atlas Vector Search	Recuperación semántica con `$vectorSearch`
Datos del agente	MongoDB (`tareas`, `asistencia`)	Estado sobre el que el agente lee y escribe
		Chat + panel de

Interfaz	Streamlit	diagnóstico de decisiones

Decisión de diseño — ciclo explícito en vez de grafo. Se evaluó usar un framework de grafos con supervisor y subagentes. Se descartó: el ciclo ReAct directo resuelve el mismo problema en unas 40 líneas legibles, mientras que la versión con grafo exige mantener y defender dos niveles de orquestación sin ganancia funcional para este alcance.

Decisión de diseño — modelo con function calling. Los sistemas `compound` de Groq, usados en la EP1, no aceptan herramientas definidas por el usuario. Fue necesario migrar a un modelo que soporte function calling.

3. Herramientas

Ocho herramientas, agrupadas según las tres capacidades que pide el encargo:

Herramienta	Tipo	Función
`buscar_material`	Consulta	Búsqueda semántica sobre el corpus institucional
`buscar_en_web`	Consulta	Fuente externa cuando el material interno no alcanza
`listar_tareas`	Consulta	Tareas y evaluaciones registradas, filtrables por estado
`estado_asistencia`	Consulta	Porcentaje por ramo y alerta bajo el mínimo exigido
`crear_tarea`	Escritura	Registra tarea con ramo, fecha y ponderación
`mover_tarea`	Escritura	Transiciona entre `por_hacer`, `en_progreso` y `hecho`
`registrar_asistencia`	Escritura	Registra asistencia o inasistencia del día
`planificar_semana`	Razonamiento	Ordena pendientes por prioridad

El docstring de cada función es funcional, no decorativo: es exactamente el texto que el modelo lee para decidir cuándo invocarla.

4. Memoria y recuperación de contexto

Corto plazo. Ventana acotada del historial de la conversación abierta (`MAX_HISTORY_MSGS`), que permite resolver referencias entre turnos como “esa” o “lo anterior”.

Largo plazo. La colección `conversaciones` en MongoDB persiste cada mensaje con su `session_id`. El agente lee los mensajes de sesiones anteriores y los inyecta como contexto del sistema, de modo que la continuidad sobrevive al cierre de la aplicación.

Recuperación semántica. `buscar_material` vectoriza la consulta con el modelo E5 y ejecuta `$vectorSearch` contra los documentos indexados, devolviendo los fragmentos con su fuente y score para

trazabilidad.

5. Planificación y toma de decisiones

El agente ajusta su comportamiento en dos niveles.

Decisión de ruta: ante cada consulta elige qué herramientas usar y en qué orden, pudiendo encadenar varias en un mismo turno. Ejemplo verificado: ante “ya terminé el taller, ¿qué sigue?”, ejecuta `mover_tarea` y luego `planificar_semana`.

Criterio de priorización: `planificar_semana` calcula la prioridad como `ponderación / días restantes`. Es determinista y está en Python, no delegado al modelo, de modo que el resultado es reproducible y auditable. Una evaluación de 35% a dos días obtiene mayor prioridad que una de 20% a veinte días.

El ciclo está acotado por `MAX_PASOS_AGENTE` para evitar bucles indefinidos, y los fallos de herramienta se devuelven al modelo como texto en lugar de interrumpir la ejecución.

6. Evidencia de pruebas

Suite	Archivo	Qué valida
Ciclo del agente	`evaluation/test_ciclo_mock.py`	Encadenamiento, tope de pasos y manejo de fallos, con modelo simulado y sin consumo de API
Casos funcionales	`evaluation/run_agente.py`	Diez casos contra el agente real; snapshot con PASS/FAIL versionado

Los casos funcionales cubren escritura, consulta, razonamiento, encadenamiento, consulta fuera de dominio, dato faltante (que el agente pregunte la fecha en vez de inventarla) y continuidad conversacional.

Resultado de la ejecución del 2026-10-22: 9/10 PASS.

Caso	Tipo	Herramientas invocadas	Resultado
AG-01	escritura	crear_tarea	PASS
AG-02	consulta	listar_tareas	PASS
AG-03	razonamiento	planificar_semana	PASS
AG-04	escritura	registrar_asistencia	PASS
AG-05	consulta	estado_asistencia	PASS
AG-06	rag	buscar_material, buscar_en_web	FAIL
AG-07	encadenado	planificar_semana, listar_tareas, mover_tarea, mover_tarea, planificar_semana	PASS
AG-08	fuera_de_dominio	(ninguna)	PASS
AG-09	dato_faltante	(ninguna)	PASS
AG-10	memoria_corto_plazo	crear_tarea	PASS

El único caso fallido, AG-06, requiere `sentence-transformers`, que no dispone de wheel precompilado para la versión de Python del equipo de desarrollo. El agente detectó el fallo de la herramienta y recurrió a la fuente externa, entregando igualmente una respuesta correcta: el sistema se degrada de forma controlada en lugar de interrumpirse.

La traza del caso AG-07 ilustra el encadenamiento y la decisión adaptativa: `planificar_semana` → `mover_tarea` → `planificar_semana`, es decir, el agente consultó el estado, lo modificó y volvió a planificar sobre el estado actualizado.

7. Restricciones y deuda técnica

- **Límite del plan gratuito.** El agente carga en cada llamada el esquema de herramientas además del contexto. Se redujeron `TOP_K` y `MAX_HISTORY_MSGS` para operar dentro del límite por minuto.
- **Sincronización de la interfaz.** El panel de herramientas académicas heredado de la EP1 persiste su estado en `localStorage` del navegador. El agente opera sobre la capa de datos en MongoDB; unificar ambas fuentes queda como trabajo pendiente y declarado.
- **Alcance de usuario.** Los documentos registran un identificador de estudiante, pero la aplicación opera hoy con un único usuario.

8. Conclusión

> *Pendiente — a redactar por el equipo.*

9. Reflexiones individuales

> *Pendiente — una por integrante.*

10. Referencias

Duoc UC. (s. f.). *Oferta académica.* https://www.duoc.cl/oferta-academica/carreras/

Groq. (s. f.). *Supported models — GroqCloud documentation.* https://console.groq.com/docs/models

LangChain. (s. f.). *Tool calling.* https://python.langchain.com/docs/concepts/tool_calling/

Lewis, P., Perez, E., Piktus, A., et al. (2020). Retrieval-augmented generation for knowledge-intensive NLP tasks. *Advances in Neural Information Processing Systems, 33.*

MongoDB. (s. f.). *Atlas Vector Search.* https://www.mongodb.com/products/platform/atlas-vector-search

Wang, L., Yang, N., Huang, X., et al. (2024). Multilingual E5 text embeddings: A technical report. *arXiv:2402.05672.*

Yao, S., Zhao, J., Yu, D., et al. (2023). ReAct: Synergizing reasoning and acting in language models. *International Conference on Learning Representations.*